

Prompting and Fine-tuning

Today's agenda

1. Strategies for adapting your model to a task
 - a. Prompting
 - b. Finetuning, e.g. LoRA
2. Distillation

Prompting

From GPT to GPT-2 to GPT-3

Recall...

GPT
117M

Trained on ~1B tokens
June 2018

GPT-2
1.5B

Trained on ~14B tokens
Feb 2019

GPT-3
175B

Trained on ~300B tokens
May 2020

GPT-3: In-context learning

Use in-context examples to specify or *demonstrate* the task to be performed.

input

thanks -> merci

hello -> bonjour

mint -> menthe

otter ->

output

loutre

GPT-3: In-context learning

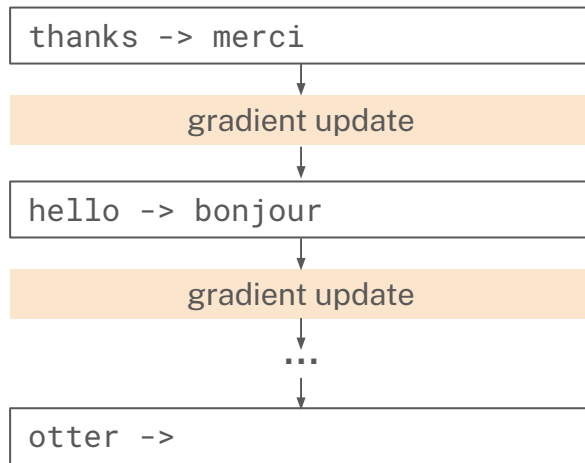
Prompt-based model **adaptation!**

No gradient updates, just prepend examples before your target input.

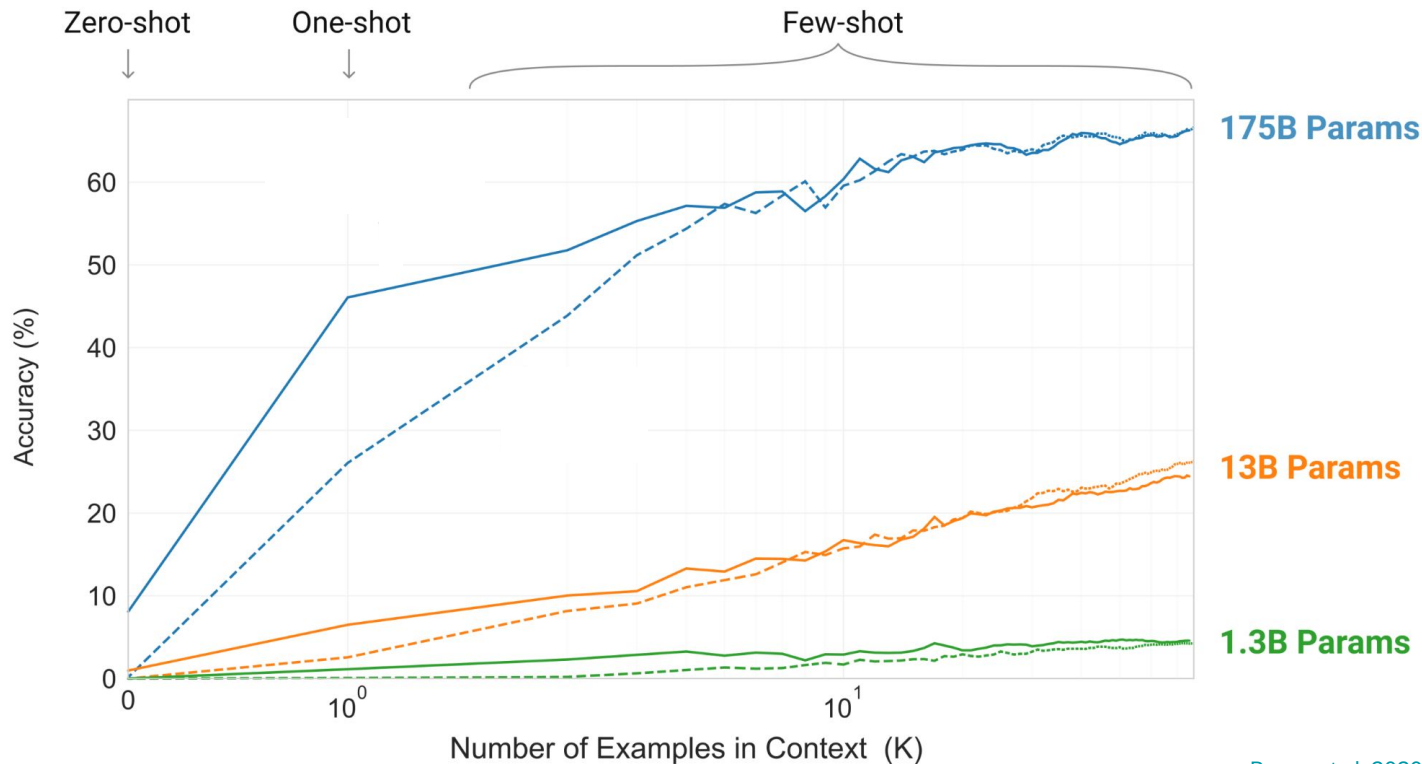
prompting

```
Translate English to French:  
thanks -> merci  
hello -> bonjour  
mint -> menthe  
otter ->
```

finetuning



An emergent property of model scale



Does in-context learning *learn*?

Demonstrations

Review: Circulation revenue has increased by 5% in Finland.

Sentiment: Positive

Review: Paying off the national debt will be extremely painful.

Sentiment: Negative

Review: The company anticipated its operating profit to improve.

Sentiment:

Positive ✓

Random demonstrations

Review: Circulation revenue has increased by 5% in Finland.

Sentiment: Neutral

Review: Paying off the national debt will be extremely painful.

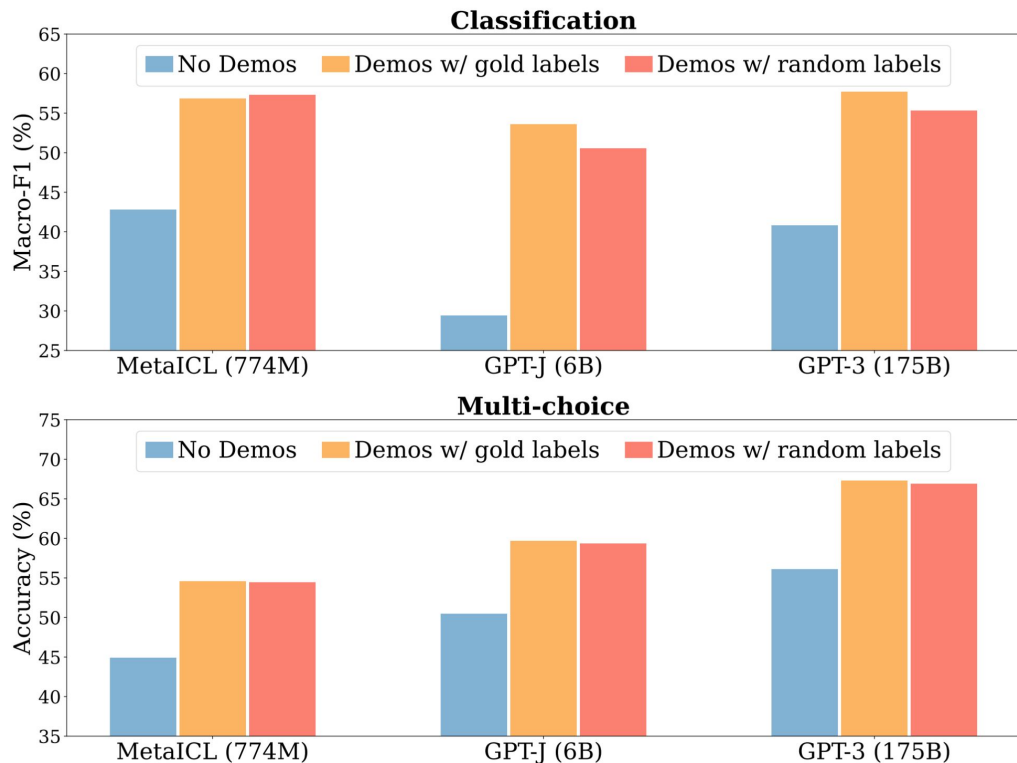
Sentiment: Positive

Review: The company anticipated its operating profit to improve.

Sentiment:

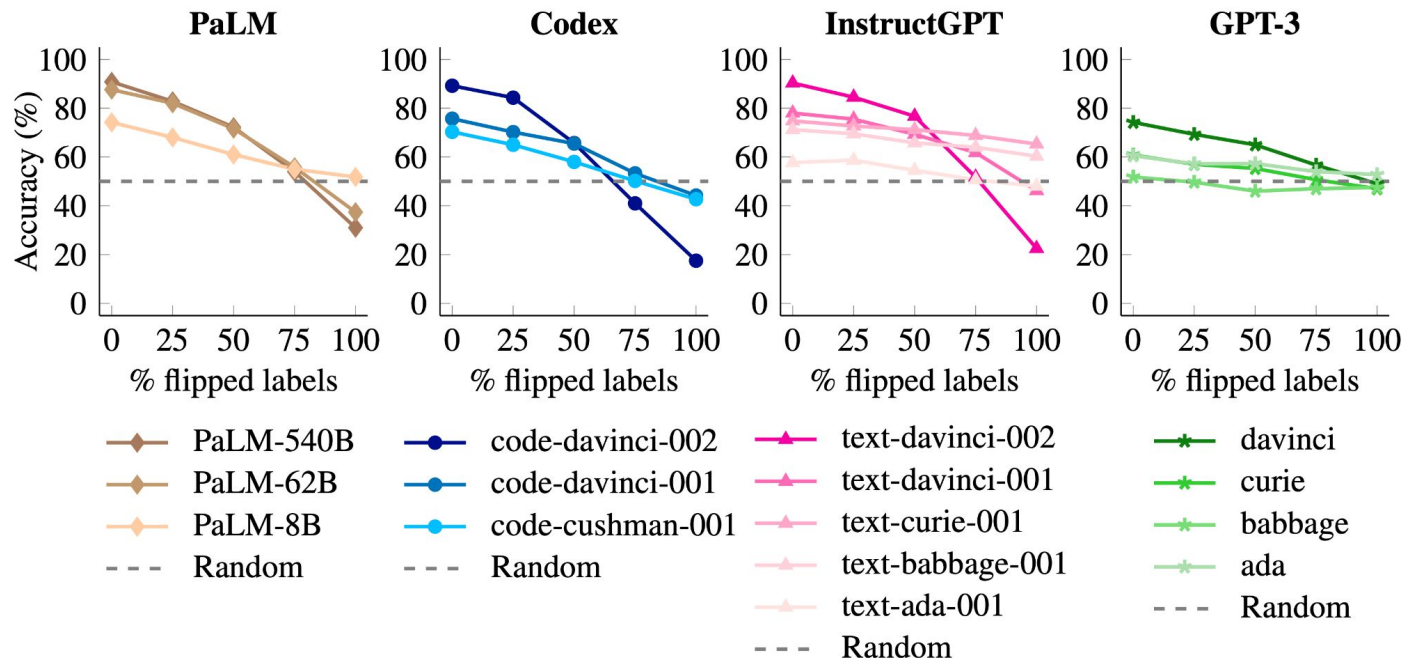
Positive ✓ 🤔

Does in-context learning *learn*?



Hypothesis: labeled few-shot data might just teach task format, and the actual correspondence or relationship between input and output is learned during pre-training.

Does in-context learning *learn*?



↓
Bigger models
use examples
to override
their priors
more!

Chain-of-thought prompting

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The answer is 27. ✗

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

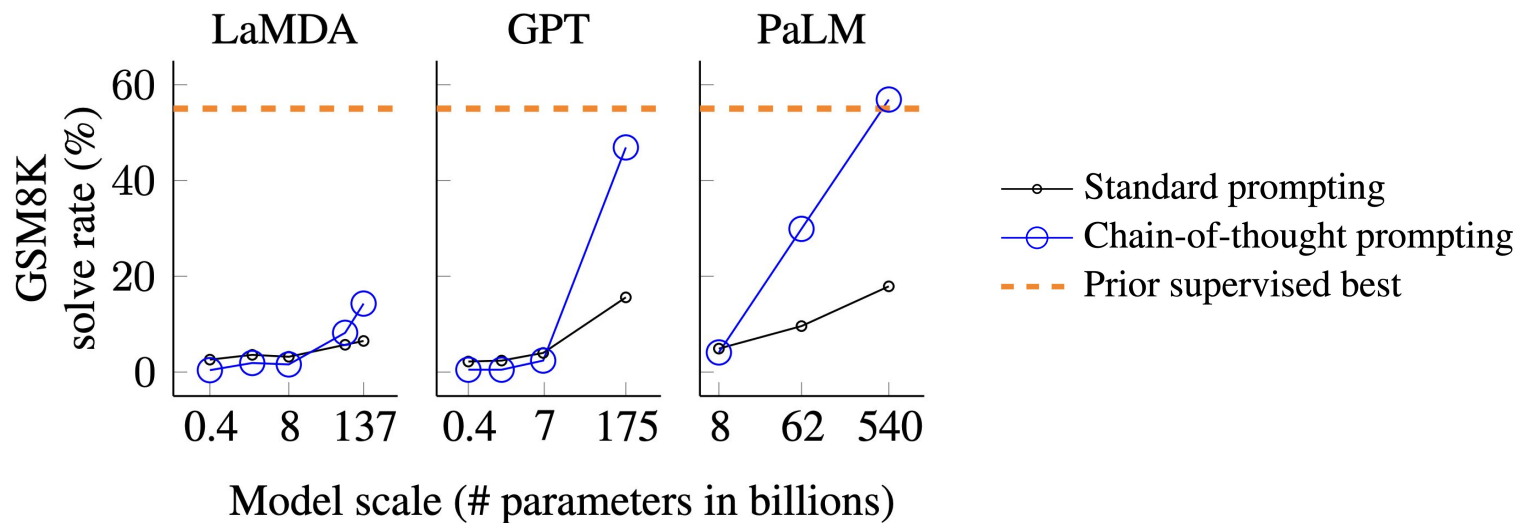
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9.

Another emergent property of model scale

Larger models make the most out of CoT!



Zero-shot CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: Let's think step by step.

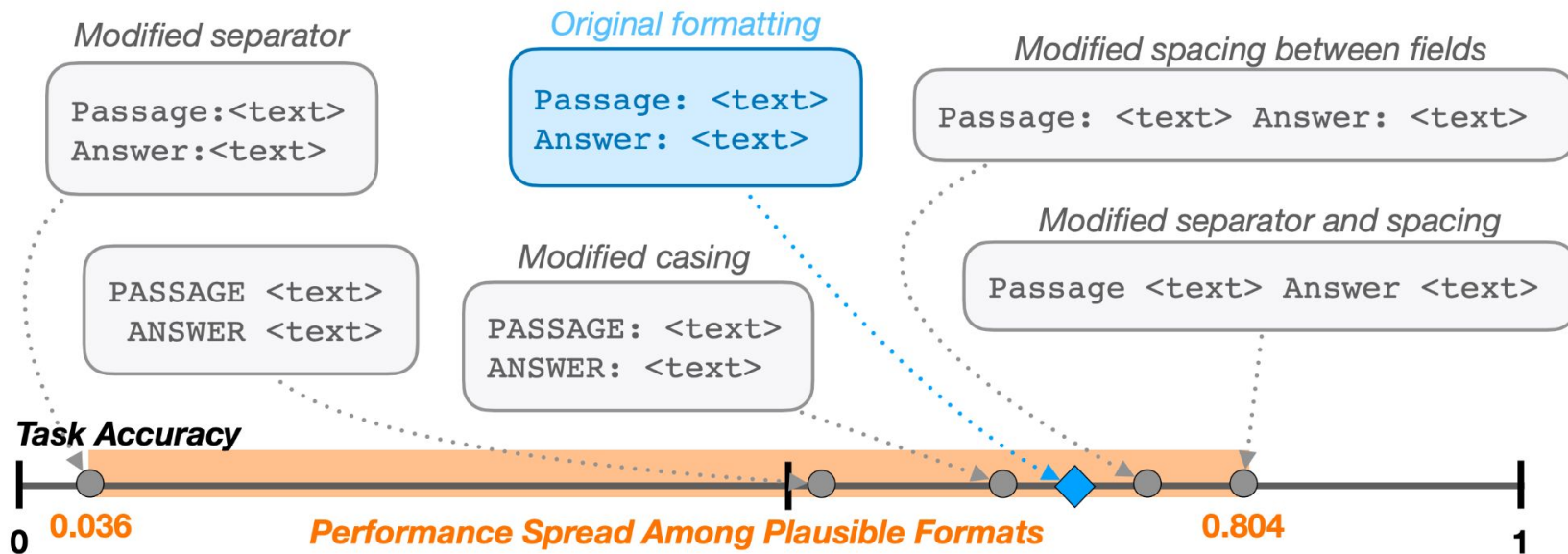
The cafeteria started with 23 apples. They then used 20 to make lunch, so $23 - 20 = 3$. They bought 6 more apples, so they now have $3 + 6 = 9$. The answer is 9.

[Kojima et al. 2022](#)

Prompt engineering

	Zero-shot CoT Trigger Prompt	Accuracy
LM-designed	Let's work this out in a step by step way to be sure we have the right answer.	82.0
Human-designed	Let's think step by step. (*1)	78.7
	First, (*2)	77.3
	Let's think about this logically.	74.5
	Let's solve this problem by splitting it into steps. (*3)	72.2
	Let's be realistic and think step by step.	70.8
	Let's think like a detective step by step.	70.3
	Let's think	57.5
	Before we dive into the answer,	55.7
	The answer is after the proof.	45.7
	(Zero-shot)	17.7

Sensitivity and inconsistency



Sensitivity and inconsistency

 **Task** Find grammatical errors in the text:

There going to their house over their.

 **LLM output**



Potential issues

</> Tagging

<err>**They're**</err> going to their house
over <err>their</err>.

incorrect copy?

[] Indexing

[0:5] = err, [30:35] = err

incorrect indices?

{ } Matching

[{"their", 1}, {"there", 2}]

multiple spans?

[Semin et al. 2026](#)

Prompts may also **specify the format of outputs.**

This affects

- task performance
- efficiency

For example, for span labeling, tagging works best for small to medium LMs, but matching has shorter outputs and is cheaper and faster.

Sensitivity and inconsistency

... matters when using LM as judges and scorers!

Pointwise

- LM gives each item a score.
- But how do we make sure “7” means the same thing each time we prompt the LM?
- What if the LM just says “5” to everything?

Pairwise

- LM compares two or more items.
- But now everything is $O(n^2)$!
- Plus LMs are susceptible to position bias...

Pointwise

Score from 1-7



Pairwise

Which response is better A, B, or tie?



LLM-hacking

We replicate **37 social science data annotation tasks**.

ideology

topic

emotion

frames

stance

politeness

tone

issue

hate speech

misinformation

...

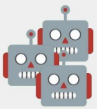


2,361 realistic hypotheses

LLMs take over the role of data annotators.

18 large language models

13 million annotations



We test the **validity of scientific conclusions**.

1.4 million regressions

LLM annotations are unreliable for downstream statistical analyses!

Task: Is manifesto *left* or *right*?
Ground truth: no effect ($p > 0.05$) 

	LLM 1, config 1	$p = 0.242$	
	LLM 1, config 2	$p = 0.001$	
	LLM 2, config 1	$p = 0.424$	
...	...	...	...

LLM hacking risk = 31% – 50%

“Virtually anything can be presented as statistically significant”

Human validation of model-based annotations can mitigate these issues!

Writing the right prompt is non-trivial

Using LMs as labelers can raise **conceptualization errors**. ([Halterman & Keith 2025](#))

These arise from incomplete or misaligned label definitions in a prompt.

Example: let's say you're a political scientist, and you want your LM to identify protests in news articles.

→ But what is a protest? Does it need at least three people? Does it include riots? Rallies? Can it be online?

All of these will lead to different LM output behavior!

In particular, **underspecification** can lead to arbitrary behavior.

Limitation of prompt-based adaptation

Sensitivity: prompt formatting, wording, etc.

- Self-consistency, e.g. taking the majority vote over multiple generations, can help, but is expensive.

Inefficiency: the full prompt needs to be processed every time the model makes a prediction.

- Have you seen my 1000-line Claude skill for generating cat stories?

Poor performance: prompting can perform worse than fine-tuning.

- Sometimes a fine-tuned RoBERTa model can outperform a commercial LLM!

Fine-tuning, e.g. LoRA

Language model fine-tuning

Given x as your input sequence and y as your output sequence for a task you care about, we calculate cross-entropy loss:

$$L_{MLE} = - \sum_{t=1}^T \log p_{\theta}(y_t | x, y_{<t})$$

Translate English to German: That is good. Das ist gut.

Only consider loss
from this part

How should we finetune?

- **Option 1:** Update only the output head (W, b)
 - Cheap
 - Assumes that our pre-trained representations are good and they're able to linearly separate the labels
- **Option 2:** Update all parameters
 - Expensive!
 - May lead to overfitting
- **Option 3:** Update a small number of parameters, somewhere
 - Cheaper
 - Parameter-efficient fine-tuning (PEFT) methods

How should we finetune?

- **Option 1:** Update only the output head (W, b)
 - Cheap
 - Assumes that our pre-trained representations are good and they're able to linearly separate the labels
- **Option 2:** Update all parameters
 - Expensive!
 - May lead to overfitting
- **Option 3:** Update a small number of parameters inside the model
 - Cheaper
 - Parameter-efficient fine-tuning (PEFT) methods

We'll just discuss one example technique, but it's a popular one that beats alternatives!

LoRA: low rank adaptation

Let's say our initial set of LM parameters are ϕ_o .

We want to find a smaller set of parameters Θ that we can use to build $\Delta\phi(\Theta)$, which we'll use to modify our original parameters ϕ_o .

The task of finding $\Delta\phi(\Theta)$ becomes optimizing over Θ :

$$\max_{\Theta} \sum_{(x,y)} \sum_{t=1}^T \log \left(P_{\phi_o + \Delta\phi(\Theta)}(y_t \mid x, y_{<t}) \right)$$

Our LM's parameters:
original ones plus some
change defined by Θ .



This is our typical fine-tuning language modeling objective, but we only update Θ , not the whole model!

Our update matrices

Let's say we have a pretrained weight matrix $W_o \in \mathbb{R}^{d \times k}$, in our original LM.

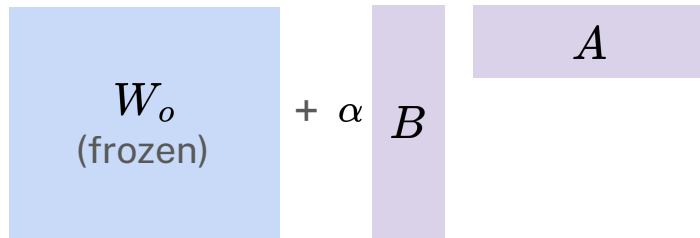
We can constrain the number of things we must update with a low-rank decomp!

Big matrices contain redundant information and can be rebuilt from smaller parts. We can rewrite a big matrix as a product of two small ones:

$$W_o + \Delta W = W_o + \alpha BA \quad \text{where} \quad B \in \mathbb{R}^{d \times r} \quad A \in \mathbb{R}^{r \times k} \quad r \ll \min(d, k)$$

Tradeoff between
pretrained knowledge
and task-specific
knowledge

Only these two have
trainable params.
Smaller matrices =
fewer params.



LoRA

Applied to attention weights in each transformer layer.

Typically, rank $r = 8, 16$, or 64 (vs. $d = 4096$ or more)

- Example: LoRA $r = 16$ of LLaMA 7B trains only 20M params (0.3% of the total).
- With higher r , LoRA converges to full FT.
- We get diminishing returns with $r > 64$, likely because the task-specific “delta” lives in a low-dimensional subspace

If we want to switch to a different task, we can get our original weights back by simply subtracting BA and adding a different set of matrices $B'A'$. Adding and removing LoRa is inexpensive!

QLoRA is a version of LoRA that uses even less memory: models are quantized to 4-bit!

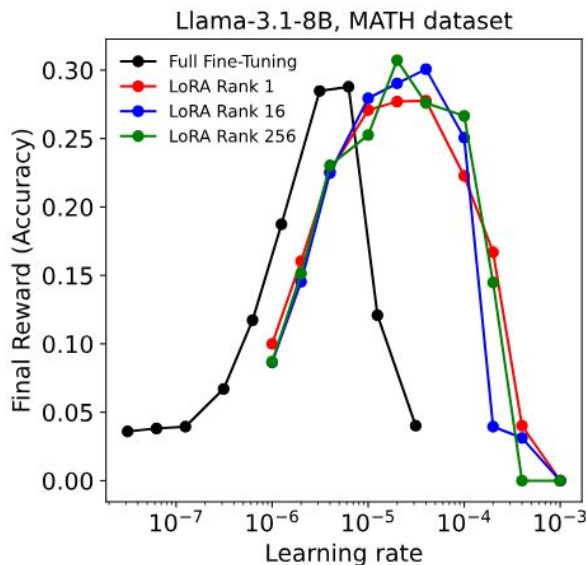
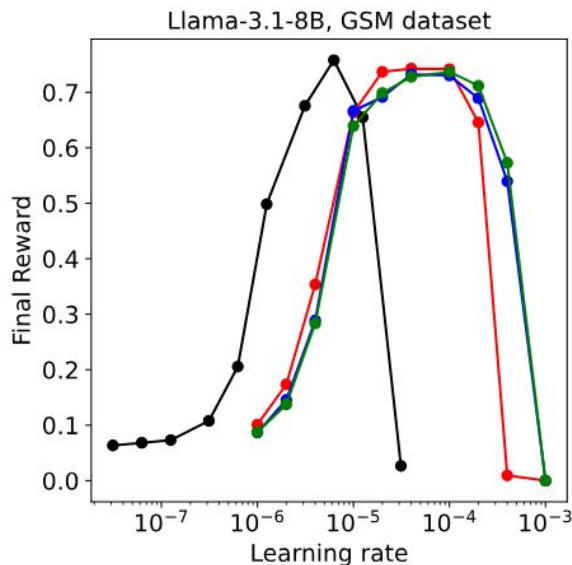
LoRA performs well

Validation accuracy is great even with rank = 1

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

LoRA performs well

Wider range of performant learning rates, and same top performance as full fine-tuning



From [Thinking Machines](#)

Effects of fine-tuning

Pretraining data covers a wider distribution of language than fine-tuning data.

So, fine-tuning narrows the space of sequence possibilities.

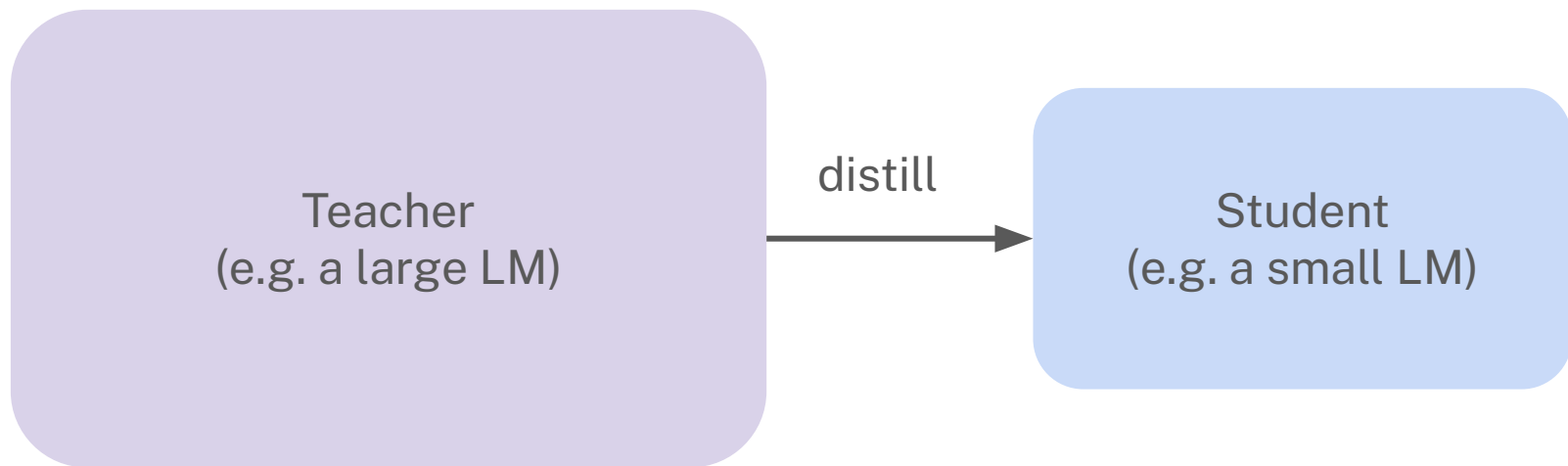
This means:

- A model fine-tuned to summarize doesn't work well on translation.
- A model fine-tuned with a specific formatting requires that formatting.
- A fine-tuned model might not be able to do in-context learning too well.

Still, fine-tuning can be done efficiently, and it can be better than prompt engineering *if* you have lots of task-specific data on hand!

Distillation

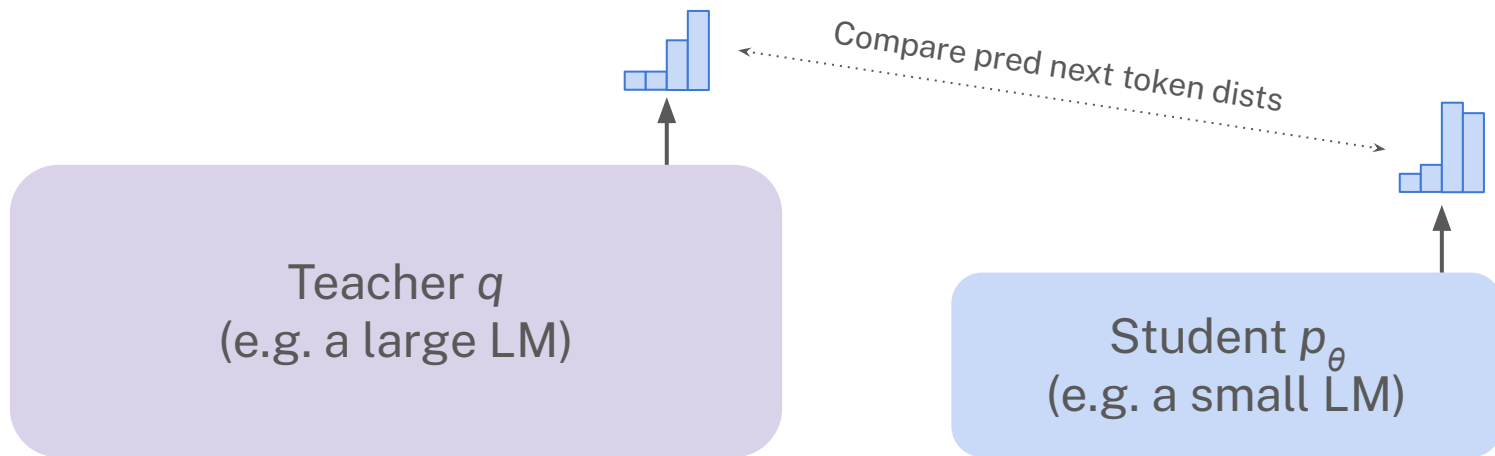
Knowledge distillation



Token-level knowledge distillation

Train a student to mimic the teacher's token distributions, w/ cross entropy loss

$$-\sum_{y_t \in V} q(y_t | y_{<t}, x) \log p_{\theta}(y_t | y_{<t}, x)$$



Token-level knowledge distillation

Train a student to mimic the teacher's token distributions, w/ cross entropy loss

$$- \sum_{y_t \in V} q(y_t | y_{<t}, x) \log p_{\theta}(y_t | y_{<t}, x)$$

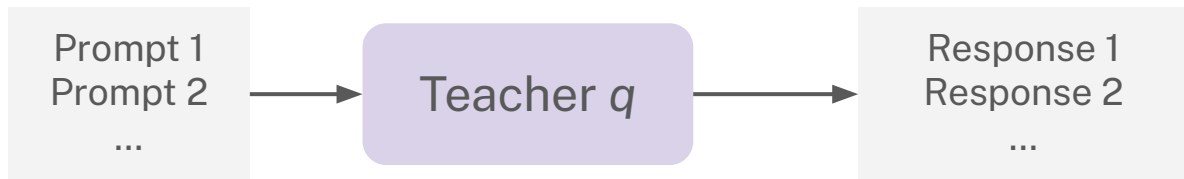
Minimize KL divergence between teacher and student:

$$\min_{\theta} \text{KL}(q(y|x) || p_{\theta}(y|x)) = \min_{\theta} \mathbb{E}_{y \sim q(y|x)} \left[\sum_t \sum_{y_t \in V} -q(y_t | y_{<t}, x) \log p_{\theta}(y_t | y_{<t}, x) \right]$$

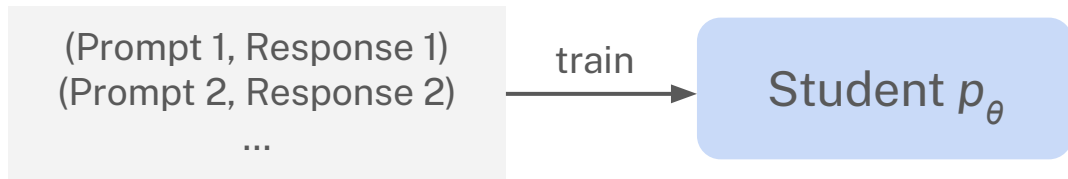
Minimize the expected loss over all output tokens

Sequence-level knowledge distillation

Generate data with a teacher



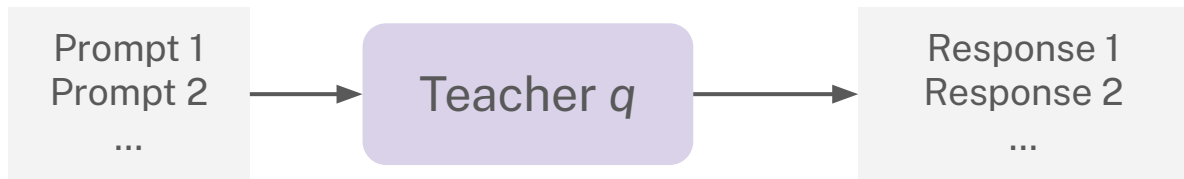
Fine-tune student on the generated data



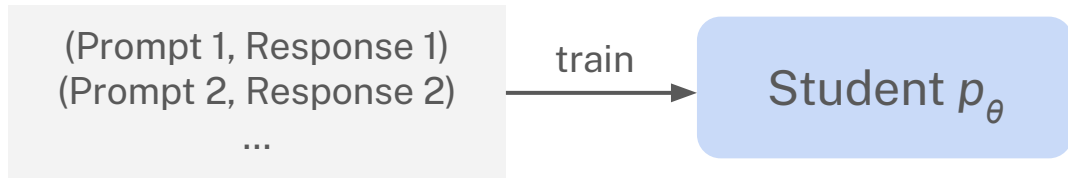
Example: DeepSeek-R1-Distill-Qwen-7B

Sequence-level knowledge distillation

Generate data with a teacher



Fine-tune student on the generated data



Example: DeepSeek-R1-Distill-Qwen-7B

Minimize KL:

$$\begin{aligned} \min_{\theta} \text{KL} (q(y|x) || p_{\theta}(y|x)) \\ = \min_{\theta} \mathbb{E}_{y \sim q(y|x)} [-\log p_{\theta}(y|x)] \end{aligned}$$

No need for the teacher's probs, just need **samples**

Sequence-level knowledge distillation

The teacher can be an “augmented” language model, e.g.

$$q \propto p_{LM}(y | x) \cdot A(x, y)$$

A classifier or verifier's output, e.g. if 0 then don't include the generation in the distillation data

If the augmented teacher q is better than p_{LM} , then the student can become better than p_{LM} after distilling from q .

Example: Phi 4 was distilled from GPT-4o's generations that were verified, self-revised, and carefully curated. ([Abdin et al. 2024](#))

Today's agenda

1. Strategies for adapting your model to a task
 - a. Prompting
 - b. Finetuning, e.g. LoRA
2. Distillation

Logistics

OHs change this week: Wed 9/30 3:00-3:30 PM, Fri 10/2 10:30-11:30 AM

Upcoming milestones:

- Peer Feedback I
- Written Project Proposal (now with an unlimited Appendix)

Practice problems!

Lucy will pass them out

If time, we will share answers and reasoning